Gen
AI

## Module - 2

# Fundamentals of GenAI

-

muhammadosama.hq@gmail.com

Gen
AI

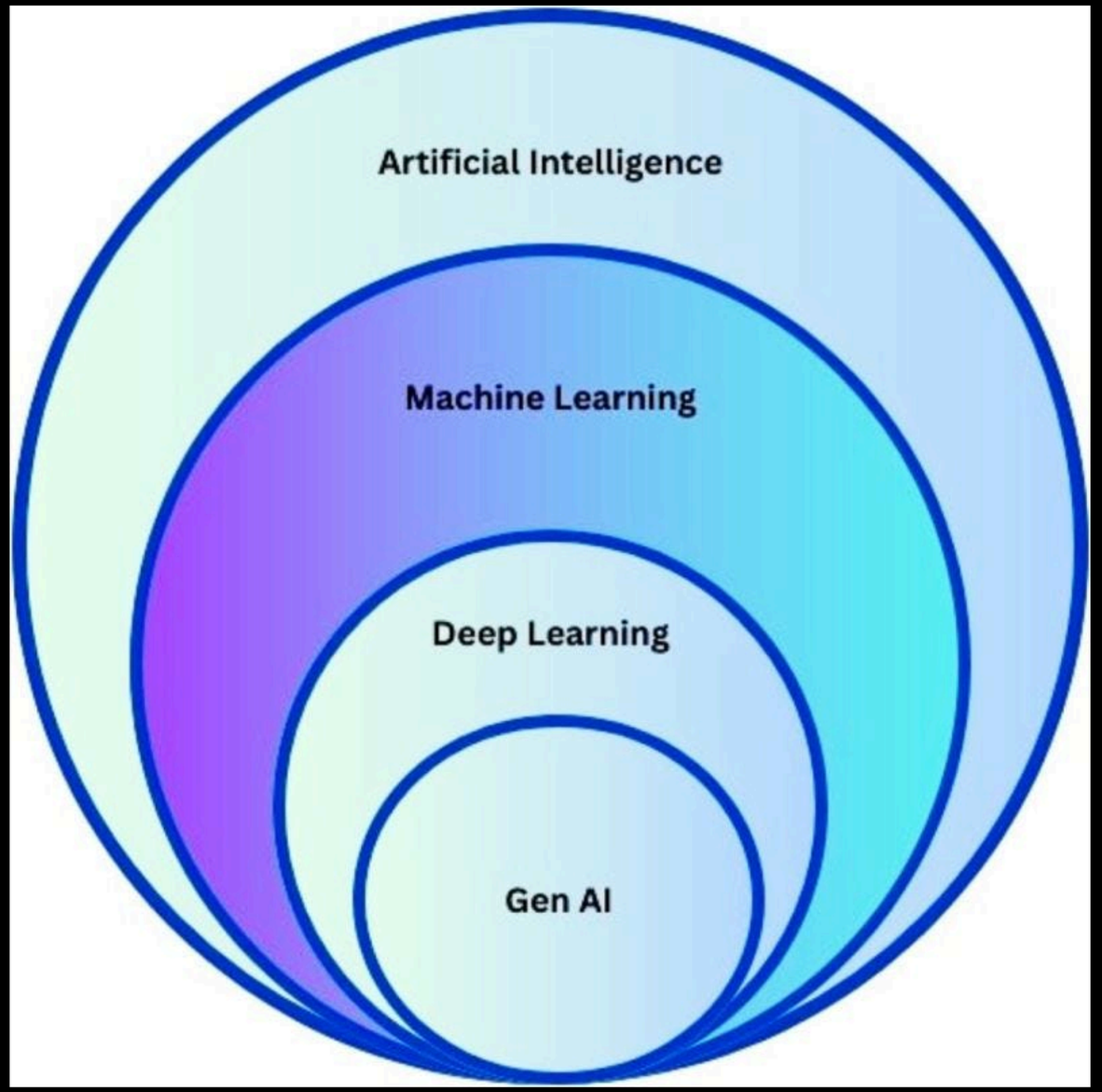
# What is Artificial Intelligence (AI)?

Artificial intelligence is the capability of computational systems to perform tasks typically associated with human intelligence, such as learning, reasoning, problem-solving, perception, and decision-making.

Source: wikipedia

**Artificial intelligence is the branch of Computer Science**

# Gen AI


Artificial Intelligence
Machine Learning
Deep Learning
Gen AI

Gen
AI

# Basics of AI

Datasets are large collections of information that AI systems use to learn. They can include things like images, paragraphs of text, or even numbers from sensors.

Training is the process of feeding data to a model so it can learn from it.

Model is a program that has been trained on a dataset

Gen
AI

# What is Generative Artificial Intelligence (GenAI)?

Gen
AI

# GenAI:

Generative AI, sometimes called gen AI, is artificial intelligence (AI) that can create original content such as text, images, video, audio or software code in response to a user's prompt or request.

Source: IBM

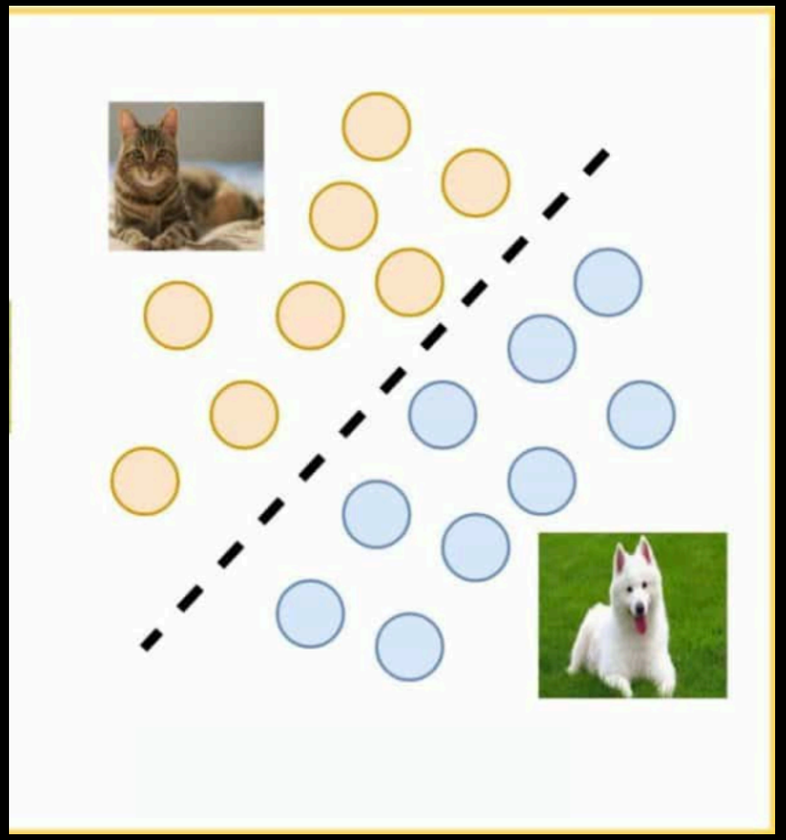
Gen
AI

# Difference between Discriminative & Generative Models

Gen
AI

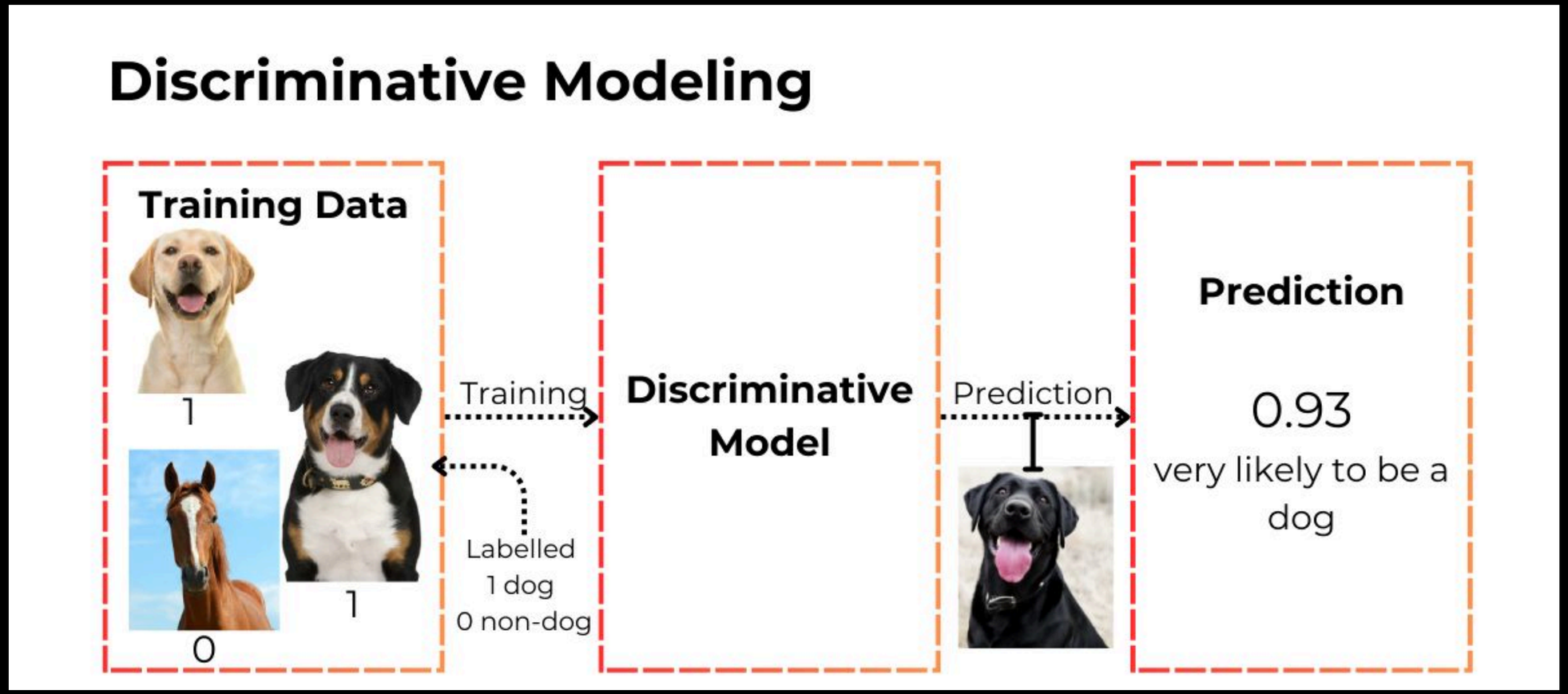
# Discriminative Models

**Discriminative** models are designed to discriminate between values of the outcome.



Gen
AI

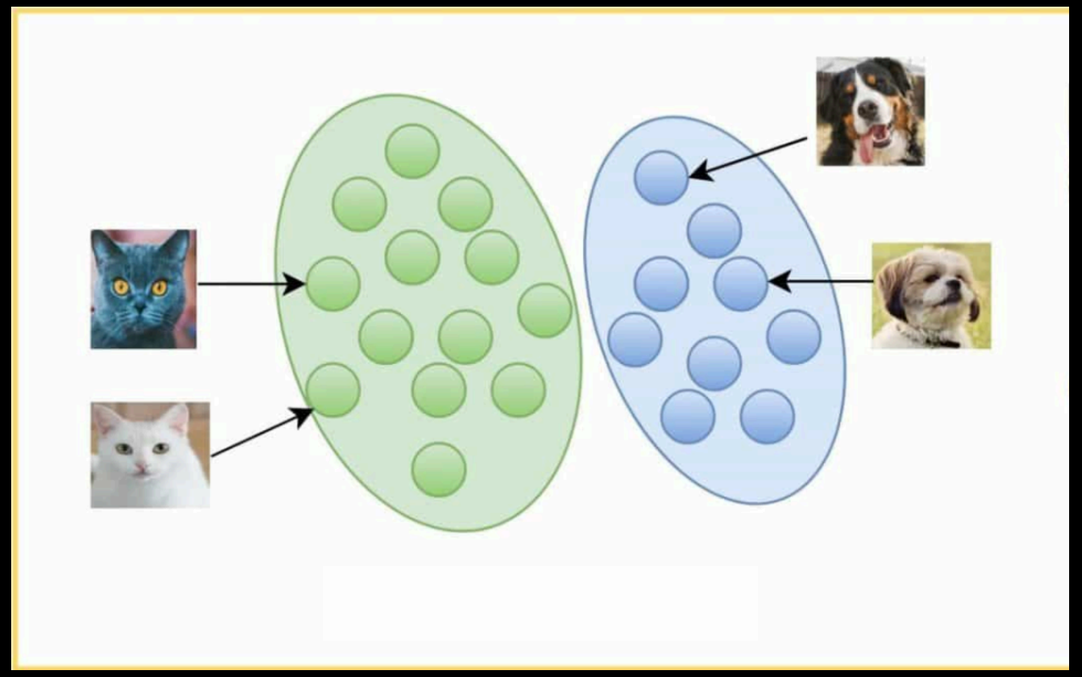
# Discriminative Model Example



Gen
AI

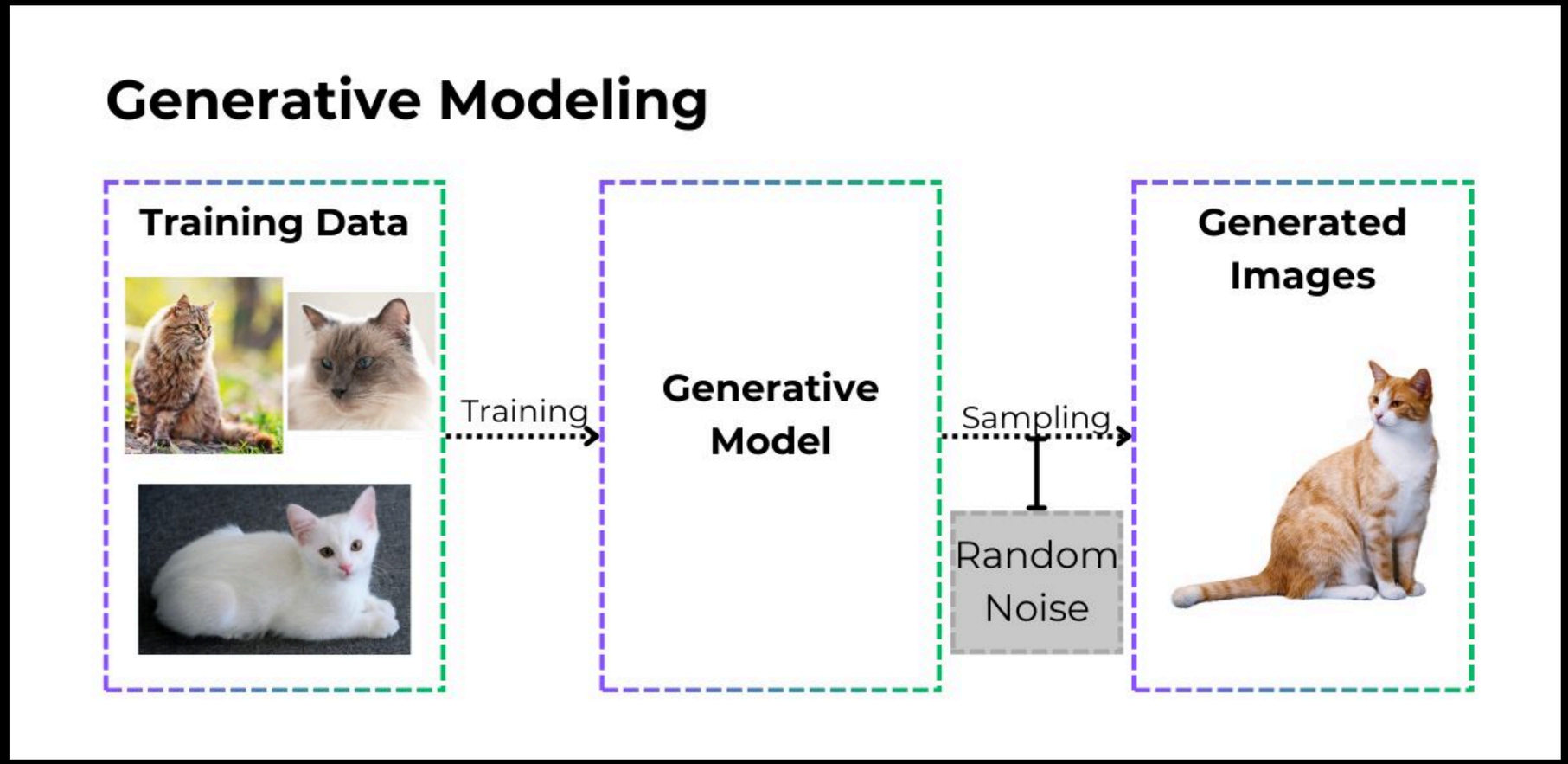
# Generative Models

**Generative** models are designed to create new data that is similar to its training data.



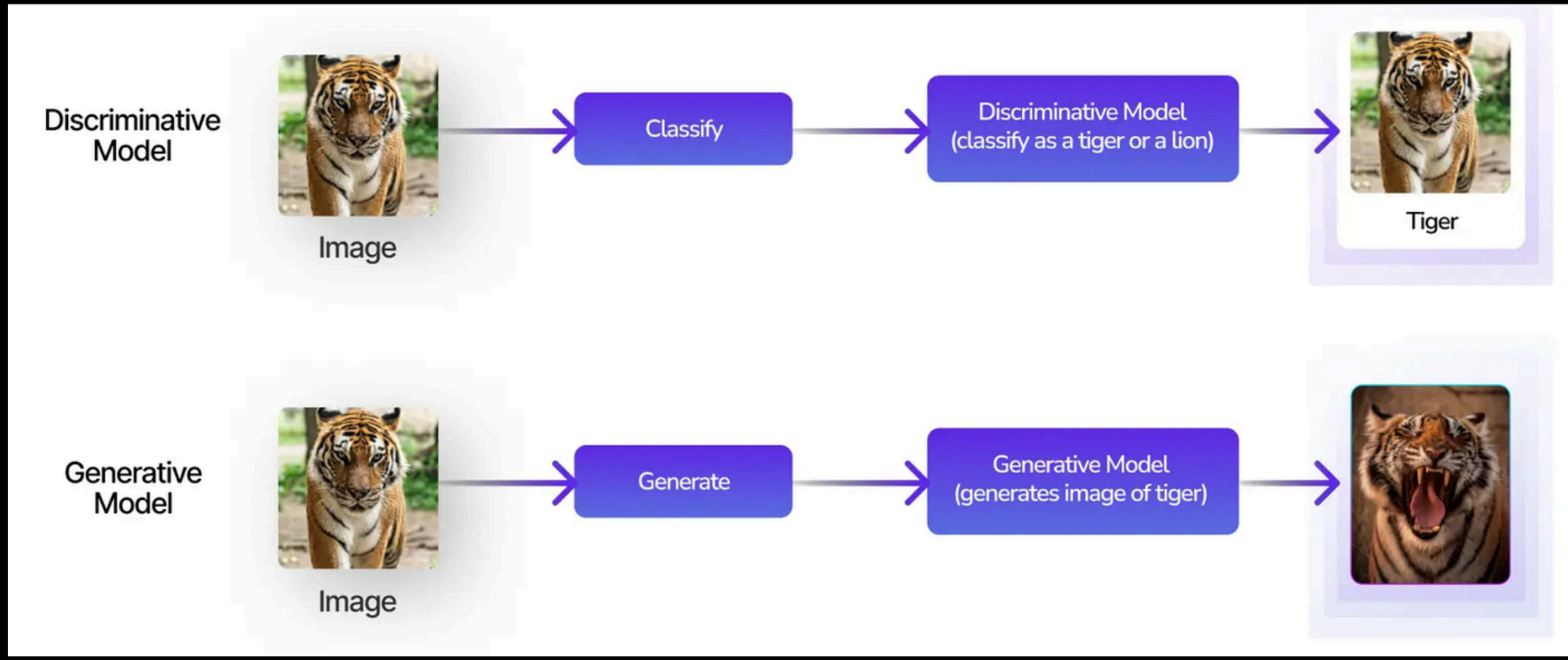
Gen
AI

# Generative Model Example



Gen
AI

# Discriminative & Generative Models



Gen
AI

# Real World AI Applications

**Can you tell which of these apps just discriminate and which actually create?**

- Gmail Spam Classification
- ChatGPT
- Midjourney
- Mobile Face Recognition Lock
- YouTube Video Recommendations
- Google Translate
- Netflix Movie Rating Prediction
- Gmail AutoComplete

Gen
AI

# Pre-Trained Model

A pretrained model is a machine learning model that has been previously trained on a large dataset.

# Fine-Tuned Model

A pre-trained model that is further trained on a smaller, task-specific dataset.

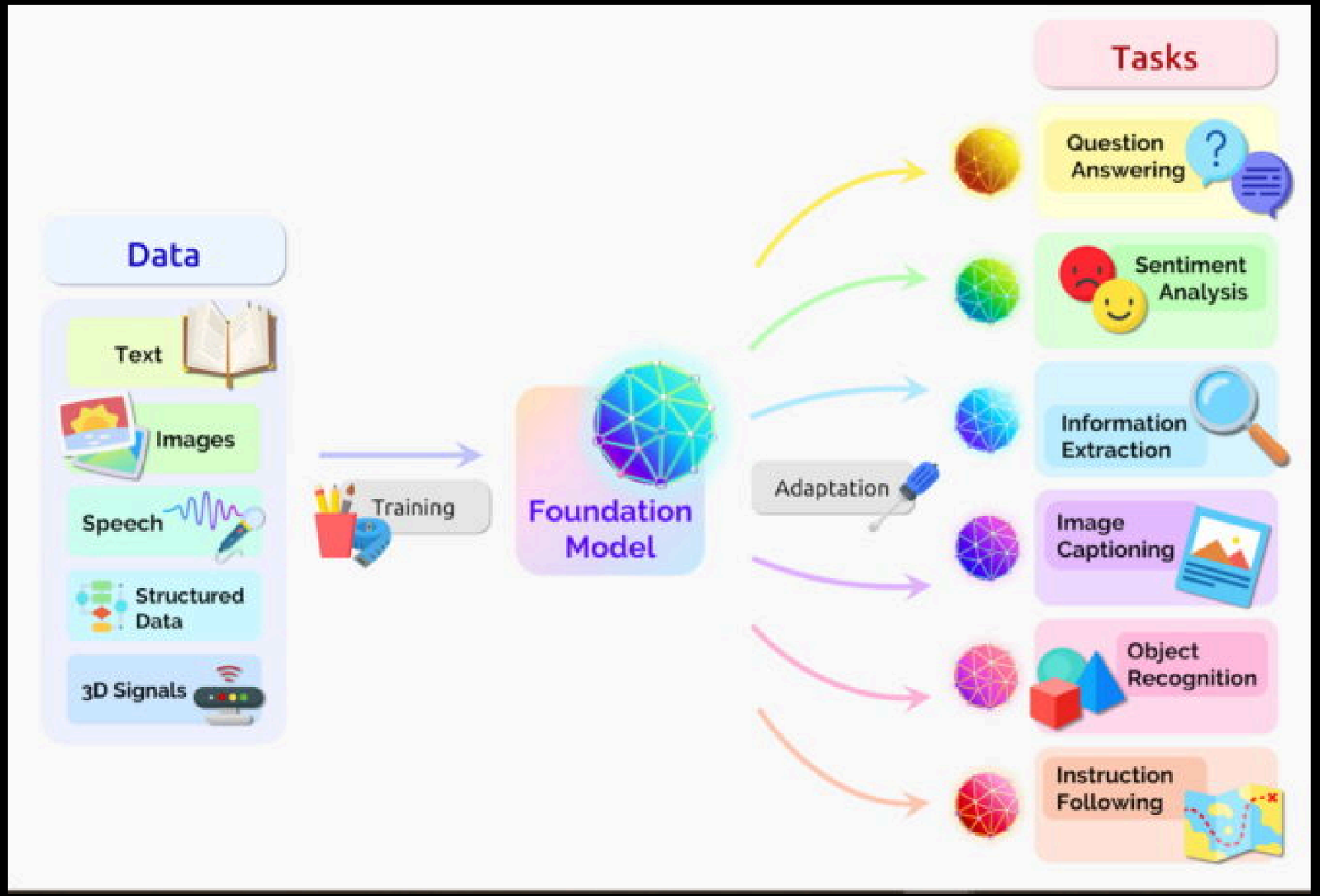
Gen
AI

# Foundation Model

A foundation model is a large-scale AI model trained on massive and diverse datasets that can be adapted to a wide range of downstream tasks through fine-tuning or prompting.

Gen
AI

# Foundation Model



# Examples of  Foundation Models

- BERT
- GPT
- Stable Diffusion
- Claude 3.5 Sonnet
- Amazon Nova

Gen
AI

# Categories of Foundational Models

- Large Language Models
- Vision Language Models
- Multimodal Large Language Models

Gen
AI

# Introduction To LLM:

A large language model (LLM) is a type of artificial intelligence that uses machine learning techniques to understand and generate human language.

Source: Redhat

# Examples of LLMs

- GPT-1 (2018)
- BERT (2018)
- GPT-2 (2019)
- GPT-3 (2020)
- T5 (Google, 2020)
- GPT-3.5 (2022)
- PaLM (2022)
- LLaMA 1 (2023)
- GPT-4 (2023)
- Claude 3 (2024)

Gen
AI

# What is VLM(Vision LanguageModel)?

A Vision-Language Model is an AI model that can understand both images and text together.

## Examples:

- GPT-4o (OpenAI)
- GPT-4V (Vision)
- Google Gemini Vision
- Claude 3 Opus / Sonnet (Vision)

Gen
AI

# What is Multimodal LLM?

A foundational language model that can understand and process more than one type of input (mode).

Gen
AI

# Let’s Make First LLM Call

Gen
AI

# First LLM Call

```python
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI()

response = client.responses.create(
    model="gpt-5.2",
    input="Write a short bedtime story about a unicorn."
)

print(response.output_text)
```

Gen
AI

# First LLM Based Application

Gen
AI

# Code Generation Application

```python
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI()

user_prompt = input("What kind of code you want to generate?\n\n")
result = client.responses.create(
    model="gpt-5.1-codex-max",
    input=f"Generate code for the following request:\n{user_prompt}"
)

print(result.output_text)
```

Gen
AI

# Introduction to LangChain

-

Gen
AI

# What is Langchain?

LangChain is a framework for developing applications powered by large language models (LLMs).

Source: LangChain

Gen
AI

# Langchain Packeges

LangChain is a framework that consists of a number of packages.

- langchain-core
- langchain
- Integration packages (like, langchain-openai, langchain-anthropic)
- langchain-community

Gen
AI

# Advantages of Langchain

- Efficient Pipeline (Chains)
- Model Agnostic (Plug & Play)
- Huge Community

Gen
AI

# Langchain Core Components

- Model
- Prompt
- Messages
- Chains (Runnables)
- Tool

Gen
AI

# Closed Source LLMs

Gen
AI

# OpenAI LLM Model

```python
from langchain_openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

llm = OpenAI(model='gpt-3.5-turbo-instruct')

result = llm.invoke("What is the capital of Pakistan")

print(result)
```

Gen
AI

# OpenAI ChatModel

```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(model='gpt-4', temperature=1.5, max_completion_tokens=30)

result = model.invoke("how many wonders are in the world?")

print(result.content)
```

Gen
AI

# Difference between LLM and Chat Model

| Feature | Large Language Model (LLM) | Chat Model |
|---|---|---|
| **Definition** | General-purpose models trained to generate and understand text. | A version of an LLM fine-tuned or wrapped for conversational use. |
| **Input format** | Plain text prompt. | Structured input (messages with roles like *user*, *assistant*, *system*). |
| **Output** | Raw text completion. | Chat-style response (dialogue-aware). |

Gen
AI

# Prompts

## Prompts are input to the llm.

### List sample prompts:

- Summarize this paragraph in 3 sentences.
- Translate this sentence into Urdu: I love learning AI.
- List 5 benefits of regular exercise.
- What is the capital of Pakistan?
- Who developed the theory of relativity?
- Compose a funny poem about coffee.

Gen
AI

# Tokens

Website: https://platform.openai.com/tokenizer

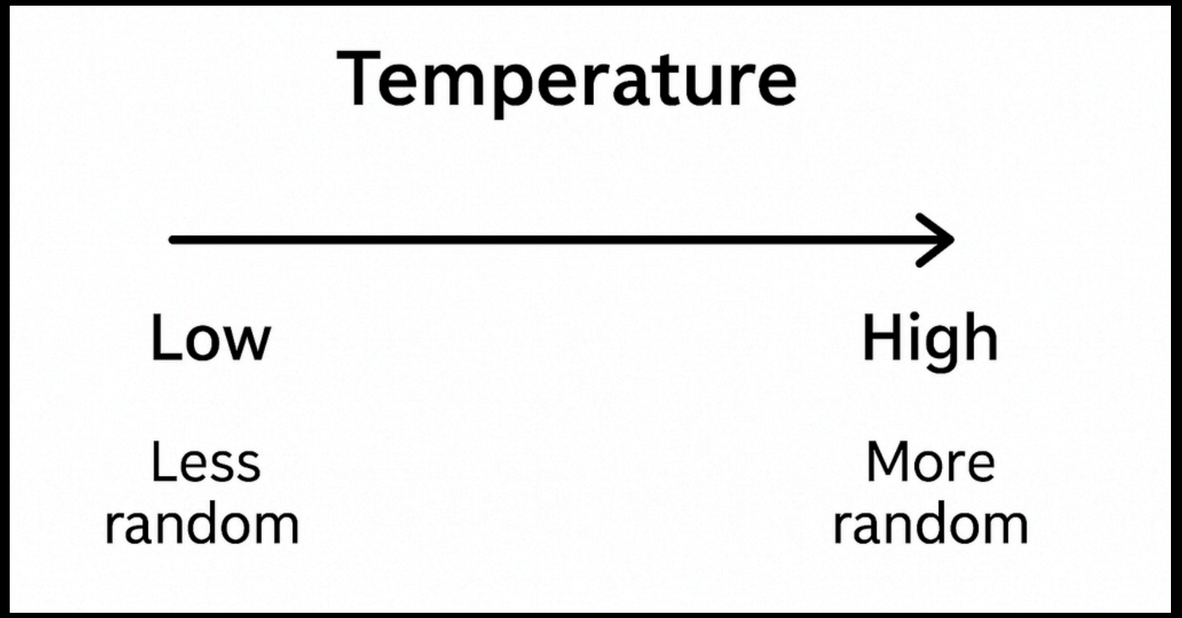
| Tokens | Characters |
| --- | --- |
| 79 | 401 |

```
Artificial-intelligence-driven hyperpersonalization in e-commerce platforms
 often relies on neurocomputationally-inspired architectures, cross-lingu
istic embeddings, and semi-autonomous sub-modules. Interestingly, words
 like “antidisestablishmentarianism,” “pseudopseudohypoparathyroidism,” and
 concatenatedstringswithoutspaces frequently split into several tokens when
 processed by modern tokenizers.
```

Gen
AI

# Temperature Param

It just controls the randomness of output Tokens



Gen
AI

# Use Case:

A student is studying geography and needs a clear, factual answer for homework.

Gen
AI

# Code Walkthrough

```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

# Low temperature for accurate, consistent facts
model = ChatOpenAI(model='gpt-4o', temperature=0, max_tokens=100)

prompt = (
    "What is the capital city of Japan? "
    "Also, name two major industries in that city."
)

result = model.invoke(prompt)
print(result.content)
```

Gen
AI

# Use Case:

A children's book publisher wants charming ideas for a new story about space animals.

Gen
AI

# Code Walkthrough

```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(model='gpt-4o', temperature=1.6, max_tokens=150)

prompt = (
    "Invent a friendly creature that lives on Mars made of stardust and music. "
    "Describe its appearance, how it communicates, and what it eats for breakfast."
)

result = model.invoke(prompt)
print(result.content)
```

Gen
AI

# init_chat_model

```python
from langchain.chat_models import init_chat_model
from dotenv import load_dotenv

load_dotenv()

model = init_chat_model("gpt-4.1")

result = model.invoke('What is the capital of Pakistan?')

print(result)
```

Gen
AI

# Context Window

The context window of an LLM (Large Language Model) is the total number of tokens the model can remember at once.

# Example

- Model: GPT-3.5
- Context window: 4,096 tokens

Gen
AI

# Hallucination Problem

Hallucinate

**Importance/Urgency:**

Messaging apps are used by 80B+ global users (Statista, 2023), yet 65% of users report frustration with “non-productive” chats (McKinsey). SMEs (90% of global businesses) need tools that bridge communication and action to stay competitive. Agentic Messaging App addresses this critical gap, merging social and task-driven communication.

Gen
AI

# Knowledge Cut Off

```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

llm = ChatOpenAI(model="gpt-4o-mini")

prompt = "Who won the World Series yesterday?"
response = llm.invoke(prompt)
print("Model response:", response.content)
```

Gen
AI

# Memory

In LangChain, memory means your app can remember what was said earlier in the conversation instead of forgetting everything after each question.

Example:

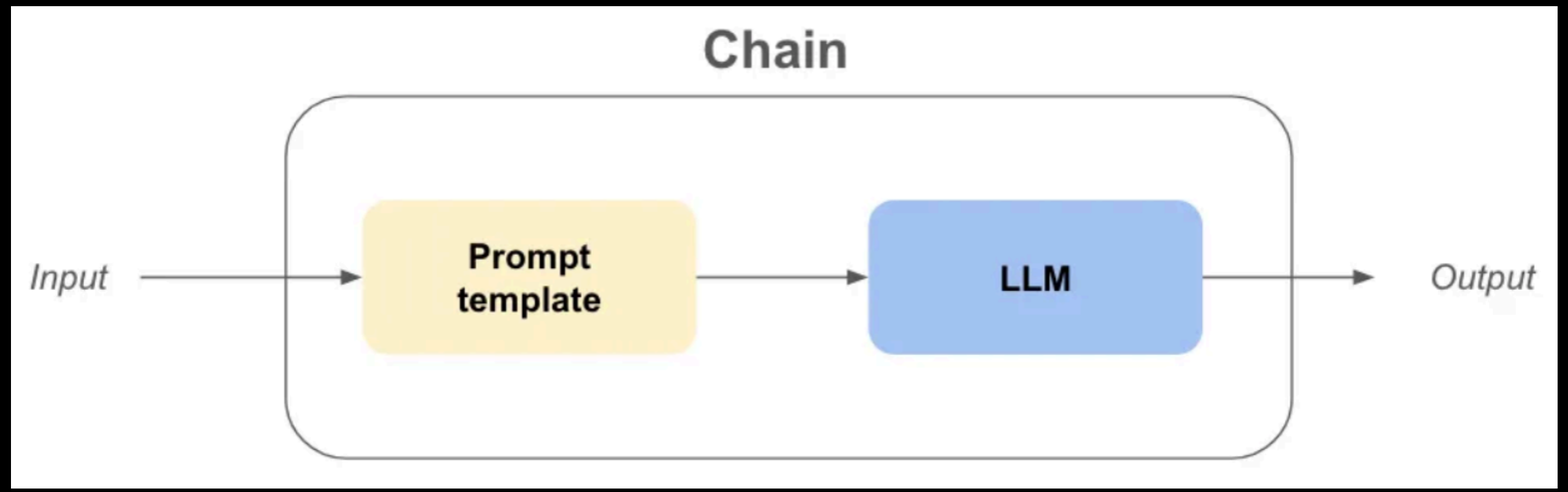
- You: “My name is Osama.”
- Later: “What’s my name?”

Gen
AI

# Chains

In LangChain, a chain is simply a pipeline of calls where:

- The output of one step becomes the input to the next.
- You can connect multiple components (models, prompts, tools, retrievers, etc.) into a workflow.



Gen
AI

# Exercise with Temperature Param

In this exercise, you will explore how the temperature parameter in LangChain affects the creativity and randomness of model outputs.

1. Use the ChatOpenAI model from LangChain.
2. Run the same prompt with the following temperature values: 0, 1, 1.5, and 2.
3. For each temperature value, run the prompt twice and record the responses.
4. Compare the differences across temperatures.

Gen
AI

# Sample Prompts

Following are the prompts that are required realistic response

- “What is the capital of Japan?”
- “How many continents are there on Earth?”
- “Convert 100 USD to Pakistani Rupees (approximate).”
- “Who was the first person to walk on the moon?”
- “What is the chemical symbol for water?”
- “What is 456 × 23?”
- “What year did World War II end?”

Gen
AI

# Sample Prompts

## Following are the prompts that are required creative response

- “Imagine humans could fly — describe a normal day in that world.”
- “Create a short dialogue between the Moon and the Sun.”
- “Invent a new fairy tale creature and explain its powers.”
- “Describe a futuristic city in the year 2500.”
- “If sadness were a color, how would you describe it?”
- “Invent a new flavor of ice cream and explain why people would love it.”
- “Design a chair that could be used in zero gravity.”
- “Imagine a smartphone for dogs — what features would it have?”

Gen
AI

# Chat Model - Anthropic

```python
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv

load_dotenv()

model = ChatAnthropic(model='claude-3-5-sonnet-20241022')
result = model.invoke('What is the capital of Pakistan?')

print(result.content)
```

Gen
AI

# Module Versioning Concept

```
# langchain

langchain[google-genai]
langchain[google-genai]>=0.3.0 # it is the langchain module version only not for extras
                               # we cant provide version for extras

# to provide versions for all
langchain>=0.3.0
langchain-google-genai==0.2.1
google-generativeai>=0.7.0,<1.0.0
```

Gen
AI

# Chat Model - Gemini

```python
from langchain_google_genai import ChatGoogleGenerativeAI
from dotenv import load_dotenv

load_dotenv()

model = ChatGoogleGenerativeAI(model='gemini-1.5-pro')
result = model.invoke('What is the capital of Pakistan?')

print(result.content)
```

Gen
AI

# Cons of Closed Sourced LLM

- API Charges
- Data Privacy
- No Fine Tuning Support

Gen
AI

# Open Source LLMs

Gen
AI

# Hugging Face Models

```python
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id="deepseek-ai/DeepSeek-V3.2",
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)
result = model.invoke("What is the capital of Pakistan?")
print(result)
```

Gen
AI

# LLM Call Methods

Gen
AI

# LLM Calls Methods

In langchain there are 3 invovation methods:

- Invoke
- Batch
- Stream

# Invoke

The most straightforward way to call a model is to use invoke() with a single message or a list of messages.

Gen
AI

# Invoke – Code - Example 1

```python
from langchain_groq import ChatGroq
from dotenv import load_dotenv
load_dotenv()
llm = ChatGroq(
    model="llama-3.1-8b-instant", # qwen/qwen3-32b
    temperature=0,
)
response = llm.invoke("Tell me the sky colour")
print(response.content)
```

Gen
AI

# Invoke – Code - Example 2

```python
from langchain.messages import HumanMessage, AIMessage, SystemMessage
from langchain_groq import ChatGroq
from dotenv import load_dotenv
load_dotenv()
llm = ChatGroq(
    model="llama-3.1-8b-instant", # qwen/qwen3-32b
    temperature=0,
)
conversation = [
    SystemMessage("You are a helpful assistant that translates English to French."),
    HumanMessage("Translate: I love programming."),
    AIMessage("J'adore la programmation."),
    HumanMessage("Translate: I love building applications.")
]
response = llm.invoke(conversation)
print(response.content)  # AIMessage("J'adore créer des applications.")
```

Gen
AI

# Batch

Batching a collection of independent requests to a model can significantly improve performance and reduce costs, as the processing can be done in parallel

Gen
AI

# Batch – Code → 1

```python
from langchain_groq import ChatGroq
from dotenv import load_dotenv
load_dotenv()
llm = ChatGroq(
    model="llama-3.1-8b-instant", # qwen/qwen3-32b
    temperature=0,
)
```

Gen
AI

# Batch – Code → 2

```
responses = llm.batch([
    "Why do parrots have colorful feathers?",
    "How do airplanes fly?",
    "What is quantum computing?"
])
for i, response in enumerate(responses):
    print(f"########## {i} ##########")
    print(response.content)
```

Gen
AI

# Stream

Most models can stream their output content while it is being generated. By displaying output progressively, streaming significantly improves user experience, particularly for longer responses.

Gen
AI

# Stream – Code - Example 1

```python
from langchain_groq import ChatGroq
from dotenv import load_dotenv
load_dotenv()
llm = ChatGroq(
    model="qwen/qwen3-32b", # llama-3.1-8b-instant
    temperature=0,
)
query = "write a 500 words essay on agentic AI"
stream = llm.stream(query)
while True:
    try:
        chunk = next(stream)
        print(chunk.content)
    except StopIteration:
        break
```

Gen
AI

# Stream – Code - Example 2

```
from langchain_groq import ChatGroq
from dotenv import load_dotenv
load_dotenv()
llm = ChatGroq(
    model="qwen/qwen3-32b", # llama-3.1-8b-instant
    temperature=0,
)
query = "write a 500 words essay on agentic AI"
for chunk in llm.stream(query):
    print(chunk.text, end="",flush=True)
```

Gen
AI

# Use Case

-

Gen
AI

# Real Time Conversational Assistant

This application is a terminal-based real-time AI assistant that enables users to interact dynamically with an intelligent system. It accepts user input directly from the terminal and generates token-by-token streaming responses, creating a natural, human-like conversational experience with minimal latency.

## Application Flow:

1. The user enters a query through the terminal.
2. The system processes the input using an LLM.
3. The response is streamed instantly to the console in real time.

Gen
AI

# Code WalkThrough → 1

```python
from langchain_groq import ChatGroq
from dotenv import load_dotenv

load_dotenv()
llm = ChatGroq(
    model="llama-3.1-8b-instant",
    temperature=0,
)
print("Real-Time AI Conversational Assistant")
print("Type 'exit' to quit.\n")
```

Gen
AI

# Code WalkThrough → 2

```
while True:
    user_query = input("You: ")

    if user_query.lower() == "exit":
        print("👋 Goodbye!")
        break

    print("AI: ", end="")
```

Gen
AI

# Code WalkThrough → 3

```
for chunk in llm.stream(user_query):
    if chunk.content:
        print(chunk.content, end="", flush=True)

print("\n")
```

Gen
AI

# Integerate UI

-

Gen
AI

# Code WalkThrough → 1

```python
import streamlit as st
from langchain_groq import ChatGroq
from dotenv import load_dotenv

load_dotenv()
llm = ChatGroq(model="llama-3.1-8b-instant", temperature=0)

st.title("Real-Time AI Conversational Assistant")
```

Gen
AI

# Code WalkThrough → 2

```
# Initialize session state
if "messages" not in st.session_state:
    st.session_state.messages = []

# Display previous messages
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.write(message["content"])

prompt = st.chat_input("Say something...")
```

Gen
AI

# Code WalkThrough → 3

```python
if prompt:
    # Save user message
    st.session_state.messages.append({"role": "user", "content": prompt})

    with st.chat_message("user"):
        st.write(prompt)
```

Gen
AI

# Code WalkThrough → 4

```
with st.chat_message("assistant"):

    response = st.write_stream(llm.stream(prompt))

# save AI message
st.session_state.messages.append(
{"role": "assistant", "content": response}
)
```

Gen
AI

# End of Class

-